

```

#
# Vadd
#
# Element wise addition of two vectors (c = a + b)
# Asks the user to select a device at runtime
#
# History: C version written by Tim Mattson, December 2009
#          C version Updated by Tom Deakin and Simon McIntosh-Smith, October 2012
#          Ported to Python by Tom Deakin, July 2013
#

# Import the Python OpenCL API
import pyopencl as cl
# Import the Python Maths Library (for vectors)
import numpy

# Import a library to print out the device information
import deviceinfo

# Import Standard Library to time the execution
from time import time
#-----

# tolerance used in floating point comparisons
TOL = 0.001
# length of vectors a, b and c
LENGTH = 1024

#-----
#
# Kernel: vadd
#
# To compute the elementwise sum c = a + b
#
# Input: a and b float vectors of length count
# Output c float vector of length count holding the sum a + b

kernelsource = """
__kernel void vadd(
    __global float* a,
    __global float* b,
    __global float* c,
    const unsigned int count)
{
    int i = get_global_id(0);
    if (i < count)
        c[i] = a[i] + b[i];
}
"""

#-----

# Main procedure

# Create a compute context
# Ask the user to select a platform/device on the CLI
context = cl.create_some_context()

# Print out device info
deviceinfo.output_device_info(context.devices[0])

# Create a command queue
queue = cl.CommandQueue(context)

# Create the compute program from the source buffer
# and build it
program = cl.Program(context, kernelsource).build()

# Create a and b vectors and fill with random float values
h_a = numpy.random.rand(LENGTH).astype(numpy.float32)
h_b = numpy.random.rand(LENGTH).astype(numpy.float32)
# Create an empty c vector (a+b) to be returned from the compute device
h_c = numpy.empty(LENGTH).astype(numpy.float32)

```

```
# Create the input (a, b) arrays in device memory and copy data from host
d_a = cl.Buffer(context, cl.mem_flags.READ_ONLY | cl.mem_flags.COPY_HOST_PTR, hostbuf=h_a)
d_b = cl.Buffer(context, cl.mem_flags.READ_ONLY | cl.mem_flags.COPY_HOST_PTR, hostbuf=h_b)
# Create the output (c) array in device memory
d_c = cl.Buffer(context, cl.mem_flags.WRITE_ONLY, h_c.nbytes)

# Start the timer
rtime = time()

# Execute the kernel over the entire range of our 1d input
# allowing OpenCL runtime to select the work group items for the device
vadd = program.vadd
vadd.set_scalar_arg_dtypes([None, None, None, numpy.uint32])
vadd(queue, h_a.shape, None, d_a, d_b, d_c, LENGTH)

# Wait for the commands to finish before reading back
queue.finish()
rtime = time() - rtime
print ("The kernel ran in", rtime, "seconds")

# Read back the results from the compute device
cl.enqueue_copy(queue, h_c, d_c)

# Test the results
correct = 0;
for a, b, c in zip(h_a, h_b, h_c):
    # assign element i of a+b to tmp
    tmp = a + b
    # compute the deviation of expected and output result
    tmp -= c
    # correct if square deviation is less than tolerance squared
    if tmp*tmp < TOL*TOL:
        correct += 1
    else:
        print ("tmp", tmp, "h_a", a, "h_b", b, "h_c", c)

# Summarize results
print ("C = A+B:", correct, "out of", LENGTH, "results were correct.")
```